Leksion
Programimi i orientuar nga objektet (OOP)

Alion Emini

kurseinformatike.com

Përmbajtja

1. Procedural vs OOP
2. Klasat
3. Puna me instancat
4. Teknika për projektimin e klasave
5. Përmbledhje

1 Procedural vs OOP

2 Klasat

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Programimi procedural

Përkufizim

Programimi procedural është stil ku fokusi është te **procedurat/funksionet** që kryejnë veprime mbi të dhënat.

- Programi ndahet në funksione: leximi i inputit, llogaritjet, lexim/shkrim skedarësh, shfaqja e rezultateve, ...
- Të dhënat ruhen në variabla dhe shpesh u kalohen funksioneve të ndryshme.
- **Ndarja** e të dhënave nga kodi që punon me to bëhet problem kur projekti rritet.

Kur proceduralja bëhet problem

- Shembull: program i madh për bazë të dhënash klientësh.
- Fillimisht:
  - emri, adresa, telefoni ruhen në 3 variabla të veçantë;
  - ti shkruan shumë funksione që marrin këta 3 parametra.
- Më vonë, struktura e të dhënave ndryshon:
  - emri, adresa, telefoni ruhen në një listë;
  - duhet të modifikosh *të gjithë* funksionet.
- Shumë punë + rrezik për futje bug-esh të reja.

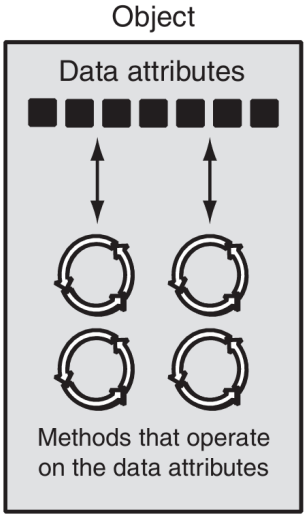
Çfarë është OOP?

Përkufizim

Programimi i orientuar nga objektet (OOP) fokusohet te **objektet** – njësi software-i që kombinojnë të dhëna + metoda që punojnë mbi ato të dhëna.

- Të dhënat quhen **attribute të të dhënave** (angl. *data attributes*).
- Funksionet brenda objektit quhen **metoda**.
- Objekti është njësi **e vetë-mjaftueshme**:
  - ruan gjendjen (atributet),
  - ofron sjellje (metodat).

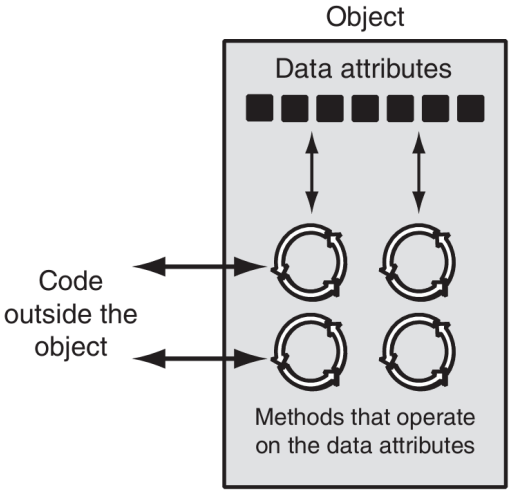
Objekti ka gjendje dhe sjellje



Inkapsulimi dhe fshehja e të dhënave

- **Inkapsulimi**:
  - kombinimi i të dhënave + kodit brenda një objekti.
- **Fshehja e të dhënave**:
  - atributet e objektit nuk aksesohen direkt nga jashtë;
  - vetëm metodat e objektit mund t'i lexojnë / modifikojnë.
- Kodi jashtë objektit ndërvepron vetëm me **metodat** e tij.
- Nëse struktura e brendshme e të dhënave ndryshon:
  - modifikojmë vetëm metodat;
  - kodi jashtë objektit nuk ndryshon.

Objekti inkapsulohet dhe fsheh të dhënat



Shembull: objekti ora me alarm (atributet)

Imagjino një orë me alarm si objekt software-i. Ajo mund të ketë atributet e mëposhtme:

- `current_second` (0–59)
- `current_minute` (0–59)
- `current_hour` (1–12)
- `alarm_time` (orë dhe minutë)
- `alarm_is_set` (`True`/`False`)

- Këto përshkruajnë **gjendjen aktuale** të orës.
- Përdoruesi nuk i prek direkt – ato janë *private*.

Shembull: ora me alarm (metodat publike)

Disa nga metodat publike të kësaj ore:

- `set_time`
- `set_alarm_time`
- `set_alarm_on`
- `set_alarm_off`

- Përdoruesi shtyp butona ⇒ thirr këto metoda.
- Metodat publike lejojnë ndryshim të kontrolluar të gjendjes.
- Këto janë metoda që mund t'i thërrasin entitetet jashtë objektit.

Shembull: ora me alarm (metodat private)

Brenda objektit ka edhe metoda **private**, p.sh.:

- `increment_current_second`
- `increment_current_minute`
- `increment_current_hour`
- `sound_alarm`

- Çdo sekondë thirret `increment_current_second`.
- Kur arrin 59, kthehet në 0 dhe thirret `increment_current_minute`, e kështu me radhë.
- Kur koha përputhet me `alarm_time` dhe `alarm_is_set` është `True`, thirret `sound_alarm`.
- Përdoruesi nuk i sheh këto detaje — janë pjesë e **funksionimit të brendshëm**.

1 Procedural vs OOP

2 Klasat
- Përkufizimet e klasave: `Coin`
- Përdorimi i klasës `Coin`
- Fshehja e atributeve
- Ruajtja e klasave në module
- Klasa `BankAccount`
- Metoda `__str__`

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Çfarë është një klasë?

Përkufizim

Klasë quajmë kodin që specifikon atributet dhe metodat për një lloj të caktuar objekti.

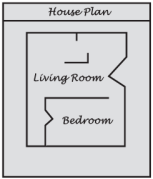
- Mund ta mendosh si një **plan** (*blueprint*) për objekte.
- Vetë klasa *nuk* është objekt, por një përshkrim i tij.
- Nga një klasë mund të krijojmë shumë objekte (**instanca** të klasës).

Analogia: plani i shtëpisë

- Plani i shtëpisë përshkruan si duket shtëpia, por nuk është vetë shtëpia.
- Mund të ndërtojmë disa shtëpi të ndryshme nga i njëjti plan.
- Po kështu, nga një klasë mund të krijojmë shumë objekte.

Analogia: plani i shtëpisë

Blueprint that describes a house

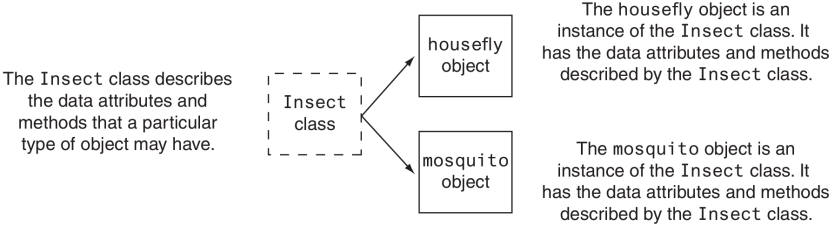


Instances of the house described by the blueprint

Analogia: prerësja e biskotave

- Një *cookie cutter* (prerëse biskotash) ka një formë.
- Me të mund të bësh shumë biskota me të njëjtën formë.
- Klasën imagjinoje si prerësen; objektet janë biskotat individuale.
- Çdo biskotë zë vend fizikisht ⇒ çdo objekt zë hapësirë në memorie.

Shembull: klasa `Insect` dhe instancat e saj



- Klasa `Insect` përshkruan çfarë kanë të përbashkët insektet.
- `housefly` dhe `mosquito` janë **instanca** të kësaj klase.
- Secili objekt ka atributet dhe metodat që përcakton klasa, por me gjendje të vetën.

1 Procedural vs OOP

2 Klasat

Përkufizimet e klasave: `Coin`

Përdorimi i klasës `Coin`

Fshehja e atributeve

Ruajtja e klasave në module

Klasa `BankAccount`

Metoda `__str__`

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Programi 1 - Klasa Coin

Qëllimi

Duam të simulojmë hedhjen e një monedhe (*Heads*/*Tails*) duke përdorur një klasë.

```
import random

class Coin:
    def __init__(self):
        self.sideup = 'Heads'

    def toss(self):
        if random.randint(0, 1) == 0:
            self.sideup = 'Heads'
        else:
            self.sideup = 'Tails'

    def get_sideup(self):
        return self.sideup
```

Struktura e klasës `Coin`

- Rreshti `class Coin:` fillon përkufizimin e klasës.
- Rregullat për emrin e klasës $\approx$ rregullat për variabla, por zakonisht fillon me **shkronjë të madhe**.
- Klasa ka 3 metoda:
  - `__init__` - inicializon atributet;
  - `toss` - simulon hedhjen e monedhës;
  - `get_sideup` - kthen anën që është lart.

Parametri `self`

- Çdo metodë ka një parametër të parë të quajtur zakonisht `self`.
- Kur thirret metoda, `self` **automatikisht** referon objektin mbi të cilin po punojmë.
- Kështu metoda e di *cilat atribute* duhet të lexojë apo të ndryshojë.

Shënim

Nuk je i detyruar ta quash `self`, por është konventë standarde dhe shumë e rekomanduar.

Metoda __init__

```
def __init__(self):
    self.sideup = 'Heads'
```

- Ekzekutohet automatikisht kur krijohet një instancë e klasës.
- Përdoret për të **inicializuar** atributet e objektit.
- Këtu: çdo monedhë e re fillon me `sideup = 'Heads'`.

Renditja në klasë

Zakonisht `__init__` vendoset si metoda e parë në përkufizimin e klasës.

Metoda toss

```
def toss(self):
    if random.randint(0, 1) == 0:
        self.sideup = 'Heads'
    else:
        self.sideup = 'Tails'
```

- `random.randint(0, 1)` gjeneron një numër 0 ose 1.
- Nëse është 0 ⇒ monedha është `'Heads'`.
- Përndryshe ⇒ `'Tails'`.
- Përdor `self.sideup` për të ndryshuar gjendjen e objektit aktual.

Metoda get_sideup

```
def get_sideup(self):
    return self.sideup
```

- Metodë **aksesuese** (lexon të dhëna, nuk i ndryshon).
- E thërrasim sa herë duam të dimë cila anë është lart.
- Kodi jashtë klasës nuk prek drejtpërdrejt `sideup` (nëse e bëjmë privat).

1 Procedural vs OOP

2 Klasat
- Përkufizimet e klasave: `Coin`
- Përdorimi i klasës `Coin`
- Fshehja e atributeve
- Ruajtja e klasave në module
- Klasa `BankAccount`
- Metoda `__str__`

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Programi 2 - coin_demo1.py I

```
import random

class Coin:
    def __init__(self):
        self.sideup = 'Heads'

    def toss(self):
        if random.randint(0, 1) == 0:
            self.sideup = 'Heads'
        else:
            self.sideup = 'Tails'

    def get_sideup(self):
        return self.sideup
```

Programi 2 - coin_demo1.py II

```
def main():
    my_coin = Coin()

    print('This side is up:', my_coin.get_sideup())
    print('I am tossing the coin ...')
    my_coin.toss()

    print('This side is up:', my_coin.get_sideup())

if __name__ == '__main__':
    main()
```

Çfarë bën `my_coin = Coin()`?

Deklarata: `my_coin = Coin()` kryen dy hapa:

1. Krijon në memorie një objekt nga klasa `Coin`.
2. Thërret `__init__`:
   - `self` referon objektin e ri;
   - `sideup` vendoset në `'Heads'`.

Në fund, `my_coin` referon këtë objekt me gjendje fillestare `'Heads'`.

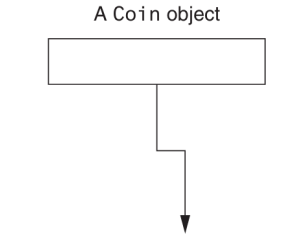
Objekti `Coin` në memorie

- Variabla `my_coin` mban një **referencë** drejt objektit.
- Objekti ka atributin `sideup`.

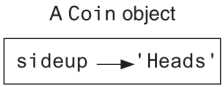
Objekti `Coin` në memorie

1. An object is created in memory from the `Coin` class.

2. The `Coin` class's `__init__` method is called, and the `self` parameter is set to the newly created object



After these steps take place, a `Coin` object will exist with its `sideup` attribute set to `'Heads'`.



Thirrja e metodave të objektit

Deklarata:

```
print('This side is up:', my_coin.get_sideup())
```

- Thërret `get_sideup` mbi objektin që referon `my_coin`.
- Python kalon automatikisht referencën te parametri i parë (`self`).

```
print('I am tossing the coin ...')
my_coin.toss()
```

- Thërritet `toss`, e cila mund të ndryshojë vlerën e `sideup`.

Shembuj output-i (coin_demo1.py)

```
This side is up: Heads
I am tossing the coin ...
This side is up: Tails
```

```
This side is up: Heads
I am tossing the coin ...
This side is up: Heads
```

```
This side is up: Heads
I am tossing the coin ...
This side is up: Tails
```

- Rezultati varet nga numri i rastësishëm i gjeneruar nga `toss`.

1 Procedural vs OOP

2 Klasat
- Përkufizimet e klasave: `Coin`
- Përdorimi i klasës `Coin`
- Fshehja e atributeve
- Ruajtja e klasave në module
- Klasa `BankAccount`
- Metoda `__str__`

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Programi 3 - coin_demo2.py (pa privatësi) I

```
import random

class Coin:
    def __init__(self):
        self.sideup = 'Heads'

    def toss(self):
        if random.randint(0, 1) == 0:
            self.sideup = 'Heads'
        else:
            self.sideup = 'Tails'

    def get_sideup(self):
        return self.sideup
```

Programi 3 - coin_demo2.py (pa privatësi) II

```
def main():
    my_coin = Coin()

    print('This side is up:', my_coin.get_sideup())
    print('I am tossing the coin ...')
    my_coin.toss()

    my_coin.sideup = 'Heads'

    print('This side is up:', my_coin.get_sideup())

if __name__ == '__main__':
    main()
```

Mashtrimi me atributin publik

```
my_coin.sideup = 'Heads'
```

- Kjo deklaratë e ndryshon direkt atributin `sideup` nga jashtë klasës.
- Rezultati i hedhjes `toss` *injorohet*.

```
This side is up: Heads
I am tossing the coin ...
This side is up: Heads
```

Problemi

Nuk kemi më simulim korrekt të hedhjes së monedhës, sepse kodi i jashtëm mund të ndryshojë rezultatin në çdo moment.

Programi 4 - coin_demo3.py (atribut privat) I

```
import random

class Coin:
    def __init__(self):
        self.__sideup = 'Heads'

    def toss(self):
        if random.randint(0, 1) == 0:
            self.__sideup = 'Heads'
        else:
            self.__sideup = 'Tails'

    def get_sideup(self):
        return self.__sideup
```

Programi 4 - coin_demo3.py (atribut privat) II

```
def main():
    my_coin = Coin()

    print('This side is up:', my_coin.get_sideup())
    print('I am going to toss the coin ten times:')
    for count in range(5):
        my_coin.toss()
        print(my_coin.get_sideup())

if __name__ == '__main__':
    main()
```

Atribut privat `__sideup`

- Emri fillon me dy viza të poshtme: `__sideup`.
- Python bën *name mangling*: nga jashtë nuk mund të shkruajmë `my_coin.__sideup`.
- Vetëm metodat e klasës mund të ndryshojnë këtë atribut.

```
This side is up: Heads
I am going to toss the coin ten times:
Tails
Heads
Heads
Tails
Tails
```

Fshehja e të dhënave

Kodi jashtë klasës përdor vetëm `get_sideup` dhe `toss`. Kjo mbron gjendjen e objektit nga ndryshimet aksidentale.

1 Procedural vs OOP

2 Klasat
- Përkufizimet e klasave: `Coin`
- Përdorimi i klasës `Coin`
- Fshehja e atributeve
- Ruajtja e klasave në module
- Klasa `BankAccount`
- Metoda `__str__`

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Programi 5 - coin.py (moduli)

```python
import random

class Coin:
    def __init__(self):
        self.__sideup = 'Heads'

    def toss(self):
        if random.randint(0, 1) == 0:
            self.__sideup = 'Heads'
        else:
            self.__sideup = 'Tails'

    def get_sideup(self):
        return self.__sideup
```

Programi 6 - coin_demo4.py

```
import coin

def main():
    my_coin = coin.Coin()
    print('This side is up:', my_coin.get_sideup())

    print('I am going to toss the coin ten times:')
    for count in range(10):
        my_coin.toss()
        print(my_coin.get_sideup())

if __name__ == '__main__':
    main()
```

Emra të kualifikuar dhe ripërdorimi

- Klasa `Coin` ruhet në modulin `coin.py`.
- Në kod tjetër:
  - `import coin`
  - krijojmë objekt me `coin.Coin()`
- **Emër i kualifikuar**: `coin.Coin`.
- Kjo mënyrë na lejon të ripërdorim klasat në shumë programe.

1 Procedural vs OOP

2 Klasat
- Përkufizimet e klasave: `Coin`
- Përdorimi i klasës `Coin`
- Fshehja e atributeve
- Ruajtja e klasave në module
- Klasa `BankAccount`
- Metoda `__str__`

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Programi 7 - bankaccount.py

```
class BankAccount:
    def __init__(self, bal):
        self.__balance = bal

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if self.__balance >= amount:
            self.__balance -= amount
        else:
            print('Error: Insufficient funds')

    def get_balance(self):
        return self.__balance
```

Analiza e klasës `BankAccount`

- Atribut privat: `__balance`.
- `__init__(self, bal)`:
  - `bal` është bilanci fillestar;
  - ruhet në `__balance`.
- `deposit(self, amount)`:
  - shton `amount` te `__balance`.
- `withdraw(self, amount)`:
  - kontrollon nëse ka mjaft fonde;
  - në të kundërt shfaq `'Error: Insufficient funds'`.
- `get_balance(self)`:
  - kthen bilancin aktual.

Programi 8 - account_test.py

```
import bankaccount

def main():
    start_bal = float(input('Enter your starting balance: '))
    savings = bankaccount.BankAccount(start_bal)

    pay = float(input('How much were you paid this week? '))
    print('I will deposit that into your account.')
    savings.deposit(pay)

    print(f'Your account balance is ${savings.get_balance():,.2f}.')

    cash = float(input('How much would you like to withdraw? '))
    print('I will withdraw that from your account.')
    savings.withdraw(cash)

    print(f'Your account balance is ${savings.get_balance():,.2f}.')

if __name__ == '__main__':
    main()
```

Shembull ekzekutimi (account_test.py)

```
Enter your starting balance: 1000.00
How much were you paid this week? 500.00
I will deposit that into your account.
Your account balance is $1,500.00.
How much would you like to withdraw? 1200.00
I will withdraw that from your account.
Your account balance is $300.00.
```

```
Enter your starting balance: 1000.00
How much were you paid this week? 500.00
I will deposit that into your account.
Your account balance is $1,500.00.
How much would you like to withdraw? 2000.00
I will withdraw that from your account.
Error: Insufficient funds
Your account balance is $1,500.00.
```

Çfarë ndodh në account_test.py?

- Lexohet bilanci fillestar ⇒ `start_bal`.
- Krijohet objekt:
  - `savings = bankaccount.BankAccount(start_bal)`
- Depozitohet rroga me `savings.deposit(pay)`.
- Shfaqet bilanci me f-string dhe `savings.get_balance()`.
- Tërhiqen para me `savings.withdraw(cash)`.

1 Procedural vs OOP

2 Klasat
- Përkufizimet e klasave: `Coin`
- Përdorimi i klasës `Coin`
- Fshehja e atributeve
- Ruajtja e klasave në module
- Klasa `BankAccount`
- Metoda `__str__`

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Programi 9 - bankaccount2.py

```
class BankAccount:
    def __init__(self, bal):
        self.__balance = bal

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if self.__balance >= amount:
            self.__balance -= amount
        else:
            print('Error: Insufficient funds')

    def get_balance(self):
        return self.__balance

    def __str__(self):
        return f'The account balance is ${self.__balance:,.2f}'
```

Programi 10 - account_test2.py

```python
import bankaccount2

def main():
    start_bal = float(input('Enter your starting balance: '))
    savings = bankaccount2.BankAccount(start_bal)

    pay = float(input('How much were you paid this week? '))
    print('I will deposit that into your account.')
    savings.deposit(pay)
    print(savings)

    cash = float(input('How much would you like to withdraw? '))
    print('I will withdraw that from your account.')
    savings.withdraw(cash)
    print(savings)

if __name__ == '__main__':
    main()
```

Si punon __str__?

- __str__ kthen një **string** që përshkruan gjendjen e objektit.
- Thirret automatikisht kur:
  - objekti i kalon print-it;
  - objekti i kalon funksionit str.

```
account = bankaccount2.BankAccount(1500.0)
message = str(account)
print(message)
```

- Këtu str(account) thërret account.__str__().

1 Procedural vs OOP

2 Klasat

3 Puna me instancat
- Metodat aksesuese dhe modifikuese
- Kalimi i objekteve si argumente

4 Teknika për projektimin e klasave

5 Përmbledhje

Atribute instance

Përkufizim

Çdo **instancë** e një klase ka setin e vet të atributeve të të dhënave. Atributet që krijohen si `self.x` quhen **atribute instance**.

- Dy objekte të së njëjtës klasë kanë *atribute me emra të njëjtë*, por vlera të ndryshme.
- Ndryshimi i atributit në një objekt nuk prek objektet e tjera.

Programi 11 - coin_demo5.py

```
import coin

def main():
    coin1 = coin.Coin()
    coin2 = coin.Coin()

    print('I have two coins with these sides up:')
    print(coin1.get_sideup())
    print(coin2.get_sideup())

    print('I am tossing both coins ...')
    coin1.toss()
    coin2.toss()

    print('Now here are the sides that are up:')
    print(coin1.get_sideup())
    print(coin2.get_sideup())

if __name__ == '__main__':
    main()
```

Shembull output-i (coin_demo5.py)

```
I have two coins with these sides up:
Heads
Heads
I am tossing both coins ...
Now here are the sides that are up:
Tails
Heads
```

- coin1 dhe coin2 janë dy objekte të ndryshme.
- Çdo objekt ka atributin e vet __sideup.
- Thirrja e toss mbi njërin nuk ndikon tjetrin.

1 Procedural vs OOP

2 Klasat

3 Puna me instancat
- Metodat aksesuese dhe modifikuese
- Kalimi i objekteve si argumente

4 Teknika për projektimin e klasave

5 Përmbledhje

Getters dhe setters

- **Metodë aksesuese** (*accessor*):
  - kthen vlerën e një atributi;
  - *nuk* e ndryshon atë.
  - Emërtim i zakonshëm: `get_...`
- **Metodë modifikuese** (*mutator*):
  - ndryshon vlerën e një atributi;
  - mund të bëjë edhe verifikime (p.sh. jo vlera negative).
  - Emërtim i zakonshëm: `set_...`

Pse janë të dobishme?

Klasa kontrollon **si** dhe **kur** mund të ndryshojë gjendja e objektit. Kodi jashtë klasës nuk prek direkt atributet private.

Shembull: klasa `CellPhone` (konceptuale)

Supozojmë klasën `CellPhone` me atributet private:

- `__manufact`
- `__model`
- `__retail_price`

Metodat:

- Getters: `get_manufact`, `get_model`, `get_retail_price`
- Setters: `set_manufact`, `set_model`, `set_retail_price`

- Kod jashtë klasës ndryshon çmimin vetëm me `phone.set_retail_price(299.99)`.

Terminologji

Shënim

- Metodat `set_...` quhen shpesh **setters** ose **mutators**.
- Metodat `get_...` quhen shpesh **getters** ose **accessors**.

1 Procedural vs OOP

2 Klasat

3 Puna me instancat
- Metodat aksesuese dhe modifikuese
- Kalimi i objekteve si argumente

4 Teknika për projektimin e klasave

5 Përmbledhje

Funksione që marrin objekte

```
def show_coin_status(coin_obj):
    print('This side of the coin is up:',
          coin_obj.get_sideup())
```

- `coin_obj` është parametër që merr një objekt `Coin`.
- Mund të thërrasim metodat e objektit brenda funksionit.

```
my_coin = coin.Coin()
show_coin_status(my_coin)
```

- Në fakt kalojmë një **referencë** drejt objektit.

Programi 12 - coin_argument.py

```
import coin

def main():
    my_coin = coin.Coin()
    print(my_coin.get_sideup())
    flip(my_coin)
    print(my_coin.get_sideup())

def flip(coin_obj):
    coin_obj.toss()

if __name__ == '__main__':
    main()
```

Analiza e coin_argument.py

```
Heads
Tails
```

```
Heads
Heads
```

- `my_coin` krijohet me `'Heads'`.
- `flip(my_coin)` i kalon të njëjtin objekt funksionit `flip`.
- Brenda `flip`, `coin_obj` dhe `my_coin` referojnë të njëjtin objekt.
- Thirrja `coin_obj.toss()` mund të ndryshojë gjendjen e objektit ⇒ ndryshimi shihet edhe jashtë.

1 Procedural vs OOP

2 Klasat

3 Puna me instancat

4 Teknika për projektimin e klasave
- Unified Modeling Language (UML)
- Gjetja e klasave në një problem
- Përcaktimi i përgjegjësive

5 Përmbledhje

1 Procedural vs OOP

2 Klasat

3 Puna me instancat

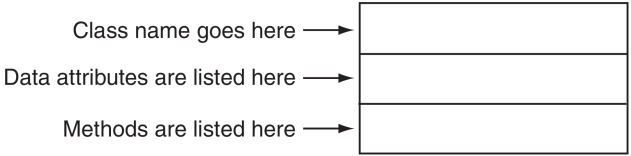
4 Teknika për projektimin e klasave
- Unified Modeling Language (UML)
- Gjetja e klasave në një problem
- Përcaktimi i përgjegjësive

5 Përmbledhje

Çfarë është UML?

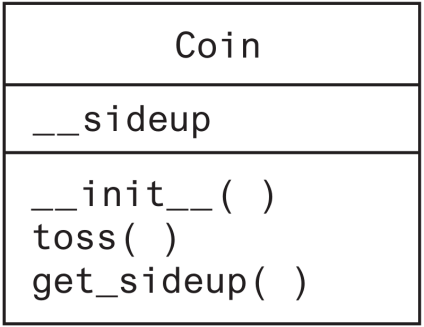
- **UML** (*Unified Modeling Language*) është një gjuhë standarde për të vizatuar sisteme OOP.
- Për klasat përdorim **diagramet e klasave**:
  - kutia ndahet në 3 pjesë:
    - emri i klasës;
    - atributet;
    - metodat.
- Ndihmon për të planifikuar klasat para se të shkruajmë kodin.

Struktura e përgjithshme e një diagrami UML



- Sipër: emri i klasës.
- Mes: atributet e të dhënave.
- Poshtë: metodat (pa parametër `self`).

UML për klasën `Coin`



UML për klasën `CellPhone`

CellPhone

`__manufact`
`__model`
`__retail_price`

`__init__(manufact, model, price)`
`set_manufact(manufact)`
`set_model(model)`
`set_retail_price(price)`
`get_manufact()`
`get_model()`
`get_retail_price()`

Pse na duhen diagramet UML?

- Jepin një **pamje të qartë** të klasës:
  - çfarë të dhënash mban;
  - çfarë mund të bëjë.
- Ekipi mund të diskutojë dizajnin pa hyrë në detaje kodi.
- Më pas, këto diagrame kthehen në klasa Python.

1 Procedural vs OOP

2 Klasat

3 Puna me instancat

4 Teknika për projektimin e klasave
- Unified Modeling Language (UML)
- Gjetja e klasave në një problem
- Përcaktimi i përgjegjësive

5 Përmbledhje

Domeni i problemit

Përkufizim

Domeni i problemit është grupi i objekteve reale, roleve dhe ngjarjeve që lidhen me aplikacionin që po zhvillojmë.

- Shembull: Joes Automotive Shop që printon *preventiva shërbimi*.
- Përshkrimi përmend:
  - klientë, makina, menaxher, preventiv shërbimi, ...

Teknika e emrave (nouns)

Hapat:

1. Shkruaj një përshkrim me tekst të domenit.
2. Nxirr të gjithë **emrat** (dhe frazat emërore).
3. Çdo emër është kandidat për klasë.
4. Pastro listën duke hequr ato që nuk duhen.

Domain

Shembull

Joes Automotive Shop servison makina të huaja dhe është i specializuar për makina Mercedes, Porsche dhe BMW. Kur një klient sjell makinën në servis, menaxheri merr emrin, adresën dhe numrin e telefonit të klientit. Pastaj menaxheri përcakton markën, modelin dhe vitin e prodhimit të makinës dhe i jep klientit një preventiv shërbimi (service quote). Preventivi tregon koston e parashikuar të pjesëve, koston e parashikuar të punës, taksën e shitjes dhe totalin e parashikuar të çmimit.

Lista fillestare e emrave

Nga përshkrimi i Joes Automotive Shop dalin emra si:

- address
- BMW
- car
- cars
- customer
- estimated labor charges
- estimated parts charges
- foreign cars
- Joes Automotive Shop
- make
- manager
- Mercedes
- model
- name
- Porsche
- sales tax
- service quote
- shop
- telephone number
- total estimated charges
- year

Pastrimi i listës (1)

- Emra që nënkuptojnë të njëjtën gjë:
  - `car`, `cars`, `foreign cars` ⇒ mbajmë vetëm `car`.
  - `Joes Automotive Shop` dhe `shop` ⇒ mbajmë vetëm `shop` (ose asnjërin).
- Emra që nuk duhen si klasa:
  - `shop`, `manager` - nuk modelohen si klasa në këtë aplikacion të vogël.

Pastrimi i listës (2)

- Emra që përfaqësojnë shembuj konkretë, jo klasa:
  - `Mercedes`, `Porsche`, `BMW` janë marka të veçanta ⇒ shembuj të `Car`.
- Emra që janë thjesht vlera:
  - `address`, `make`, `model`, `name`, `sales tax`, `year`, ...
  - këto do të jenë atribute, jo klasa më vete.

Klasat e mbetura

Pas filtrimit mbesin:

`car`, `customer`, `service quote`

- Këto kthehen në klasat:
  - `Car`
  - `Customer`
  - `ServiceQuote`

1 Procedural vs OOP

2 Klasat

3 Puna me instancat

4 Teknika për projektimin e klasave
- Unified Modeling Language (UML)
- Gjetja e klasave në një problem
- Përcaktimi i përgjegjësive

5 Përmbledhje

Çfarë duhet të dijë / bëjë një klasë?

Për çdo klasë pyesim:

- **Çfarë duhet të dijë?**
  - këto janë **atributet e të dhënave**.
- **Çfarë duhet të bëjë?**
  - këto janë **metodat**.

Shpesh përgjigjen e kemi që në përshkrimin e problemit.

Klasa `Customer` - UML

- Duhet të dijë:
  - emrin;
  - adresën;
  - numrin e telefonit.
- Duhet të bëjë:
  - inicializim;
  - set/get për secilin atribut.

Klasa Customer - UML

```
Customer
---------------------------
__name
__address
__phone
---------------------------
__init__(name, address,
         phone)
set_name(name)
set_address(address)
set_phone(phone)
get_name()
get_address()
get_phone()
```

Programi 13 - customer.py I

```
class Customer:
    def __init__(self, name, address, phone):
        self.__name = name
        self.__address = address
        self.__phone = phone

    def set_name(self, name):
        self.__name = name

    def set_address(self, address):
        self.__address = address
```

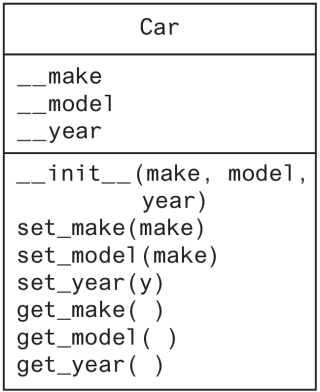
Programi 13 - customer.py II

```
    def set_phone(self, phone):
        self.__phone = phone

    def get_name(self):
        return self.__name

    def get_address(self):
        return self.__address

    def get_phone(self):
        return self.__phone
```

Klasa `Car` - UML

- Duhet të dijë:
  - markën (`make`);
  - modelin;
  - vitin e prodhimit.
- Duhet të bëjë:
  - inicializim;
  - set/get për secilin atribut.

Klasa Car - UML



Programi 14 - car.py I

```
class Car:
    def __init__(self, make, model, year):
        self.__make = make
        self.__model = model
        self.__year = year

    def set_make(self, make):
        self.__make = make

    def set_model(self, model):
        self.__model = model
```

Programi 14 - car.py II

```
    def set_year(self, year):
        self.__year = year

    def get_make(self):
        return self.__make

    def get_model(self):
        return self.__model

    def get_year(self):
        return self.__year
```

Klasa `ServiceQuote` - UML

- Duhet të dijë:
  - koston e pjesëve (`parts_charges`);
  - koston e punës (`labor_charges`).
- Duhet të llogarisë:
  - taksën e shitjes;
  - totalin e parashikuar.

Klasa ServiceQuote - UML

ServiceQuote
__parts_charges __labor_charges
__init__(pcharge, lcharge) set_parts_charges(pcharge) set_labor_charges(lcharge) get_parts_charges( ) get_labor_charges( ) get_sales_tax( ) get_total_charges( )

Programi 15 - servicequote.py I

```
TAX_RATE = 0.05

class ServiceQuote:
    def __init__(self, pcharge, lcharge):
        self.__parts_charges = pcharge
        self.__labor_charges = lcharge

    def set_parts_charges(self, pcharge):
        self.__parts_charges = pcharge

    def set_labor_charges(self, lcharge):
        self.__labor_charges = lcharge
```

Programi 15 - servicequote.py II

```
    def get_parts_charges(self):
        return self.__parts_charges

    def get_labor_charges(self):
        return self.__labor_charges

    def get_sales_tax(self):
        return self.__parts_charges * TAX_RATE

    def get_total_charges(self):
        return (self.__parts_charges +
                self.__labor_charges +
                self.__parts_charges * TAX_RATE)
```

Dizajni është iterativ

- Procesi:
  1. gjejmë klasat;
  2. vendosim atributet;
  3. vendosim metodat.
- Në praktikë:
  - sa më shumë kupton problemin, aq më shumë rishikon dizajnin;
  - mund të shtosh / heqësh atribute dhe metoda;
  - ndan ose bashkon klasa ekzistuese.

1 Procedural vs OOP

2 Klasat

3 Puna me instancat

4 Teknika për projektimin e klasave

5 Përmbledhje

Përmbledhje e kapitullit

- Programimi procedural vs OOP.
- Objekti = të dhëna (atribute) + metoda (sjellje).
- Inkapsulimi dhe fshehja e të dhënave mbrojnë gjendjen e objektit.
- Klasat: `Coin`, `BankAccount` dhe shembujt përkatës të programeve.
- Atribute instance dhe kalimi i objekteve si argumente.
- Getters/setters dhe metoda `__str__`.
- UML për modelim klasash.
- Teknikat për gjetjen e klasave dhe përcaktimin e përgjegjësive (`Customer`, `Car`, `ServiceQuote`).

Faleminderit për vëmendjen!

Pyetje?